Architectural Guardrails: An Opinionated Framework for Preventing Structural Decay in Greenfield Research Software

Art O'Liathain
22363092@studentmail.ul.ie
University of Limerick

December 2025

Keywords Evolutionary Algorithms · Genetic Programming · Grammar Evolution · Symbolic Regression · Concurrent Computing

List of Figures

List of Tables

1 Abstract

2 Introduction

Research software is almost always seen as a means to an end, a tool used to prove a hypothesis so a paper can be published. When you combine this mindset with the fact that roughly 90% of developers are self-taught [[1]], it leads to an epidemic of sub-optimal code within research projects.

This has cascading effects: while the code might work just well enough for one paper, it lacks longevity. This means future researchers often have to reimplement previous work just to start the next step. This is a difficult task, considering that on average only 25% of code repositories can run without issues or modifications [[2]]. It also raises an insidious question: if researchers produce results using a code artifact that cannot be reproduced, they may be producing results that cannot be verified.

This is why tools like static analysis are necessary to raise the floor of code quality. Unfortunately, these warnings are often ignored due to high volume [[3]]. Warnings are not enough; there needs to be an opinionated system that uses empirical data to accurately predict if a file will become a maintenance issue before it is added to a larger codebase. As relying on expert opinion only allows warnings as the rules must generalise across all codebases of a language.

This paper seeks to answer the question: Can opinionated, objective rules be derived from historical and static analysis on code repositories using machine learning?

To approach this, an explainable machine learning approach using decision trees and random forests [4] will be used. To combat the limitation of requiring a manually labeled dataset, an automatic approach using heuristic-based methods will be used to quickly curate a high-quantity dataset from research files sampling from over 200+ different research repositories. This allows the ML models to learn patterns specific to research software, increasing the confidence in the resulting quality gate.

3 Background Research

3.1 The current state of research software

Research software is software made to investigate new ideas and concepts. This comes with one very prominent issue, researchers who are leading the way to new concepts are generally not programmers by trade, 90% of scientists are self taught in programming [1]. Software is made without maintainability in mind only following the check of *It works on my computer*. This is best observed by [2] where over 9000 R files were analysed from the Dataverse Project (http://dataverse.org/). In this study following good practice only 25% of projects were runnable and adding in code cleaning, where dependencies were retroactively found and hardcoded paths were altered only 56% were able to be run. This is a shocking result, this result doesn't even compare if the results are consistent with the results this code produced. [2] did note that journals are beginning to mandate better quality standards for code through research software standards to particularly allow for reproducibility to ensure the results published by the papers are correct.

As 90% of researchers [1] don't have prior programming experience, they need good clear guidance on how to create and maintain a large software project. Unfortunately research software guidance are laughable as broadly they only focus on software reproducibility [5], [5], [6], [7], [8] (Need to reword this later). Reproducibility is considered such a simple part of industry software that it isn't considered a metric to hit, it is a given yet in research software is it laid out as being the metric for best practice. Interestingly with all the focus on reproducibility is no guidance on maintainability, coding standards or even basic structuring, the papers only if ever mention a vague sense of what should be done without clear guidelines or criteria on how it should be done. A deficiency that directly goes against the goals of [9] in creating long living research software.

To compound the problem more research software is made to fit a funding cycle without a care for the longevity of the codebase. Funding is fundamentally not suited for research software [9] notes that unlike commercial products where revenue grows linearly with users research grants are fixes and do not require or reward developers for user acquisition. Funding pushes research software to focus on itself with no incentives outside of a research paper output. A flawed approach which [9] does address with two alternative solutions; Commercial models in which the research software itself transitions to a self sufficient project which keeps itself alive and peer-production models which is a similar approach as open-source. I agree that both of these models have merit, they allow research software to live past the funding cycle and continue to improve over time, there is one key issue with this approach.

Addressing the core issue of research software longevity while adhering to the constraints governing research software as a whole is the gap this paper is seeking to fill. To create an opinionated static analysis tool that will ensure code is written well through objective quality gates. By automating the "quality check" process, we can raise

the floor of research software maintainability without requiring researchers to become professional developers. Static analysis provides the immediate, clear guidance that current literature lacks, ensuring that software is not just reproducible today, but maintainable and extendable for the researchers of tomorrow.

3.2 Maintainability

To raise code quality maintainability must be defined. Maintainability is a term used frequently in software engineering, there is no definite definition on what maintainability is but ISO25010 defines it as "The degree of effectiveness and efficiency with which a product or system can be modified to improve it, correct it or adapt it to changes in environment, and in requirements." It defines that the sub sections of maintainability are *Modularity, Re-usability, Analysability, Modifiability, Testability.* [10] This is a very broad definition which simply means how easy can the system be modified for change. Due to the broadness of the definition it can easily be used to bring other non functional requirements as sub requirements ie. Flexibility, Adaptability, Scalability, Fault tolerance, Learn ability. The lack of clear testable outcomes for each quality in the ISO standard leads to a conceptual overlap where maintainability becomes a 'catch-all' category for non-functional requirements, making assessing what is truly maintainable very difficult.

3.2.1 What does it mean to be maintainable?

To label code as maintainable, modularity is one key factor in the assessment. Modularity done correctly facilitates changeability through logical decomposition of functionality, it may seem simple yet if done incorrectly can have long lasting consequences on the maintainability and longevity of a codebase. D.L Parnas presented a study revealing how easily developers could fall into the pitfall of designing the system in a human like manner [11]. Parnas argued that designing systems following a flowchart of logical operations is a method that creates code that is not resilient. While it may be modular on paper it is highly coupled which impacts the maintainability of the codebase. The effect has been substantiated by [12] with a direct correlation being made between coupling and maintainability overhead. The core purpose of modularisation shouldn't be encapsulating subroutines into modules. Design decisions that are likely to change should serve as the foundational points to creating modular code, allowing subroutines to compositions of modules. Taking modularity into account will serve as a solid baseline to assess the maintainability of a codebase and files.

3.2.2 Measuring Maintainability

As maintainability is an abstract concept, various frameworks have attempted to reduce it to a concrete numerical metric. An early approach in this area is the Maintainability Index (MI), which calculates a score based on a weighted combination of Halstead Volume, cyclomatic complexity, lines of code (LOC), and comment density [13]. The formula typically utilized for this assessment is:

$$\mathrm{MI} = 171 - 5.2\ln(V) - 0.23(G) - 16.2\ln(\mathrm{LOC}) + 50\sin\left(\sqrt{2.46 \times C}\right) \tag{1}$$

Where:

- V represents **Halstead Volume**;
- G is **Cyclomatic Complexity**;
- C is the **percentage of comment lines**.

[13] validated this metric through feedback from 16 developers and empirical testing on industrial systems, including those at Hewlett-Packard and various defense contractors. This methodology provided a pragmatic tool for engineering managers to prioritize maintenance efforts by assigning a tangible value to code quality. While the MI provides a high-level overview of code density, it suffers from what can be described as "semantic blindness." Metrics such as Cyclomatic Complexity and Halstead Volume analyze the control flow and token count of a file but fail to interpret the structural intent or the relationships between components. This is referred to as blind metrics and is a deficiency in this method. Utilising blind metrics allow for efficient and quick but it opens the door to gaming the system in a sense, where developers could reach extremely maintainable scores syntactically while semantically being unmaintainable, thereby abusing the formula. Ex. A script that maintains a high MI score due to low complexity and short length, yet contain architectural flaws such as "God Objects" or tight coupling to external datasets that inhibit reuse. The formula was a product of its time: finely tuned to the teams and projects it was applied to, in the modern landscape the current formula would definitely not be accurate with how modern languages have evolved. Even though the formula could be adapted to a modern context it is clear that even a modernised version would suffer from "blind metrics" and is not something that should be used as only counting lines cannot yield results on the true quality of code.

An approach that would apply in a more modern context was proposed by Xiao et al, of identifying architectural debt through evolutionary analysis of a codebase [14]. Where architectural debt can be used as a measure of maintainability it quantifies debt through a formalized Debt Model, using regression analysis to capture the growth rate of maintenance costs over time. The paper proposed that there are four key architectural patterns that are the main proponents of ATD, Hub, Anchor Submissive, Anchor Dominant, and Modularity Violation. These patterns are all based on evo

Hub: Characterised by strong, mutual coupling where the anchor file and its members have structural dependencies in both directions and history dominance in at least one. This pattern often represents "spaghetti code" where a central file is overloaded with responsibilities.

Anchor Submissive: Occurs when members structurally depend on an anchor, but the anchor is historically submissive, changing whenever the members change.This typically indicates an unstable interface being forced to change by its clients.

Anchor Dominant: The reverse of submissive, where members structurally depend on the anchor, and the anchor historically dominates them, frequently propagating changes outward to its dependents.

Modularity Violation: Identified as the most common and expensive form of architectural debt across various projects. It is unique because it involves files that have no explicit structural dependencies but are frequently changed together in the project's commit history.

The four patterns were measured by preforming a pseudo longitudinal study on large open source codebases, using tools such as Understand and Titan to derive the relevant pattern by calculating the commit coefficient between every file. A metric which relates to the chance of a commit on one file will require a commit on another. An example would be given files A, B and C, with a commit history of {A,B} and {A,C} the commit coefficients in relation to A are there is a 100% chance if B or C is modified A will be also, but if A is modified there is only a 50% chance that B or C will be modified. This would then be called a fileset, this would be extrapolated over the entire commit history and codebase creating many filesets. This would allow the tool to extract semantic relations between files that are not apparent structurally. The study was able to compound this effect by measuring the number of commits labeled as bug fixes against the number of feature commits, this allows simple data analytics to measure the amount of maintenance debt that each fileset would have. Using this method, high maintenance filsets can be labelled and evaluated as a quantitative metric. The core value of their work is the identification of Modularity Violations: instances where files are forced to change in tandem despite having no logical reason to do so. In the "wild west" of research software, where commit messages are often uninformative, it is not possible to rely on a human to label a change as a "bug fix." However, a substitute that can can be explored is replacing missing semantic data with the sheer physical volume of change. This necessitates shifting the focus from why a file changed to how much it changed focusing more on temporal changes, leading to the study of Code Churn.

3.2.3 Code churn's impact on maintainability

Code churn is the rate of change of lines of code in a file. A file with high churn might have an incredibly high number of additions and deletions while remaining small in terms of LOC. This is an ATD hotspot when assessing points of interest in terms of maintainability in codebases. [15] is paper which assess how code code churn affects the maintainability of codebases, by assessing the code churn of files in large codebases, achieving this by calculating the sum total of lines added and deleted on every file. Supplementing this the maintainability of the code was measured using ColumbusQM probabilistic software quality model [16]. The paper was able to assess the levels of maintainability per file in proportion to the amount of code changes, drawing the conclusion that code that has high churn is code in which it is harder to maintain. This conclusion makes sense, it reaffirms that poorly architected code will need more changes which increases the maintainability burden of the code. The method in which the code churn was measured is quite useful and will be the approach taken for this study.

Unfortunately there is one key issue with this paper, the use of ColumbusQM for assessing the maintainability of the codebase. ColumbusQM is a statistical machine which

takes metrics such as lines of code, styling, coupling, number of incoming invocations etc.. Aggregates them which then computes a numerical output. The metrics such as styling are something so subjective and minor that the inclusion as one of the key metrics in the evaluation underscores the whole statistical model. Taking into account all of the evaluation metrics this is a case of blind assessment where structural relations or evolutionary dependencies are not measured and only a snapshot of the codebase is measured. Compounding on this the evaluation metric for the output of the statistical model was based on developer feedback and ideas, which is unreliable and subjective.

Taking these points into account, the key takeaway from [15] is the methodology regarding code churn, rather than its conclusions on maintainability. By identifying the intersection of Modularity Violations (Structural) and High Churn (Temporal), we can create a composite metric that avoids the "blindness" of ColumbusQM. This allows us to quantify architectural decay using only version control metadata, remaining entirely agnostic to both the quality of documentation and the subjective "style" of the researcher. Giving us the foundation of files to be used to create opinionated static analysis tooling.

3.3 Static Analysis

The transition from historical temporal analysis to active development requires a mechanism that can evaluate code in its current state, rather than its past. Static Analysis serves as this mechanism, offering a storied field of research focused on improving code quality by examining source code without execution. While the temporal analysis identifies the "symptoms" of architectural decay through commit history, static analysis allows us to identify the "pathology"—the specific structural patterns that lead to high maintenance overhead.

3.3.1 Current Methods

Modern methods for static analysis include data flow analysis, syntactic pattern matching, abstract interpretation, constraint based analysis etc… [17]. Using these methods present a significantly more detailed analysis of programs, where a move has been made from a token level analysis to a project level analysis, allowing approaches such as path analysis and reasoning about runtime behaviour. Yet even with this increase in analytic capabilities noise pollution in which there are too many error messages discouraging any actions being taking to remedy them, the same issue which plagued Lint [18] is still an issue to this day [19]. Polluting the developer experience creating friction and actively discouraging developers form using the tools as it becomes more effort to sift through and find the real bugs than fixing them normally [3]. This leads to the conclusion that the research community especially due to the lack of experience in the field [1] would have more difficulties differentiating fact from fiction and discouraging them from using static analysis at all. This concretely highlights the difficulties of modern static analysis tooling in a research context. The focus on a general one size fits all tool means that it lacks the ability to be opinionated be default. The gap this presents is one where research software would

have a bespoke set of curated rules reducing noise and guaranteeing improvements in code quality once executed.

This paper poses that the key issue with the analysers in the case of false positives is not the rules but how they were derived. Historically most academic research in software engineering has required significant empirical evidence to be considered by industry [20], yet the current landscape is dominated by expert analysed rules. The challenge of software maintainability with static analysis is elusive. 'Ghost Echoes Revealed' highlights this [[21]], critiquing the 'ghost echoes' produced by traditional expert-derived static analysis. These echoes are not objective truths, but rather technical artifacts—static rules that linger in codebases despite often failing to align with the lived experience and subjective assessments of human developers. By benchmarking machine learning models against human judgment, the study reveals a fundamental dissonance: expert rules frequently draw 'hard lines' that humans do not actually perceive as problematic. This suggests that codebase governance based on these traditional metrics is less an exercise in objective optimization and more an adherence to institutional haunting. The precedent set here is that for codebases to remain truly maintainable, we must move beyond the rigid, subjective 'opinions' of legacy experts and toward empirically validated models that reflect the nuanced, non-deterministic reality of human-centric code health."[21]

3.4 Machine learning

Machine learning enables exactly this: rather than relying on expert opinion to define what "maintainable" means, we can derive rules empirically from how developers actually behave. Machine learning is a pivotal technology within the context of this paper. It allows rigorous statistical analysis to be carried out to create objective metrics for the maintainability rules. Machine learning is based on the principle of minimising loss, creating a method of training an algorithm to optimise for the lowest loss through gradient descent allows for the code to be self improving within the problem domain. There are many approaches and models available but due to an emphasis put on human readability decision trees and random forest best fits this use case.

3.4.1 Decision Trees

Decision trees are a standout choice when it comes to an interpretable machine learning models. They consistently rank highly within classifier models while still supporting rule extraction which aids readability. Decision trees represent classifiers as hierarchical IF-THEN rules, where each root-to-leaf path corresponds to a conjunction of feature thresholds that directly maps to executable static analysis checks. A concrete example of this would be IF nesting > 3 THEN unmaintainable, in the case of nesting being over 3 then the code would be labeled unmaintainable. This simplistic example would be scaled up much larger with many more decision and leaf nodes creating a classifier based off simple decisions at each step. Naturally there are some drawbacks to this approach, most prominently is the overfitting problem. An overfitted model reflects the structure of the training data set too closely. Even though a model appears to be accurate on

training data,, it may be much less accurate when applied to a current data set [22]. This issue restricts their usage in current maintainability prediction methods, [23] shows that decisions trees can prove useful in a baseline analysis of maintainability predictors. Despite limitations, DT rules provide the simple interpretability missing from random forests, enabling manual validation of learned static checks against domain knowledge. Pruning or ensemble averaging in random forests addresses overfitting to a degree while retaining path-derived rules for analysis [24].

3.4.2 Random Forest

Random forest is an approach derived from decision trees and arguably one of the most popular classifier machine learning approaches. Relying on creating a set of decision trees (called a forest) each of which vote on the outcome to choose a class for the classifier, it creates a method that leverages the law of large numbers to deal with the overfitting problem in decision trees [25] in addition to increasing accuracy. The method in which overfitting can be largely ignored is that there are many trees voting, every one could potentially be overfitted yet the whole forest remains generalised to the problem as every tree would be overfitted to a different feature. Random forests average better results theoretically [25] and practically in the field of maintainability research [23] over decision trees. The key drawbacks to random forests are both the increased computational costs and the black box view when it comes to readability. Modern research [4] has made progress in creating methods of rule extraction for random forests as before the size of the forests rendered them as a black box approach. [4] focused on deriving a rule ensemble based on the calculated weighted importance of the trees. Allowing for a dimension reductionality method to be applied to the forest. Within the study a factor of 300x reduction to the trees per class was observed while retaining a 93% accuracy compared to the full forests 95% accuracy. Leveraging this rule extraction method for random forests is pivotal to create a rule set that properly generalises across projects while retaining the core idea and performance of the model and keeping with the core ideal of readability.

3.4.3 Applications to maintainability detection

The application of machine learning to maintainability detection enables a data-driven approach to identifying code structures that hinder long-term quality and evolution. Traditional static analysis methods rely on human-curated heuristics, which often fail to generalise across diverse projects or languages. In contrast, machine learning can infer maintainability rules directly from empirical evidence, allowing patterns of poor or high maintainability to emerge statistically rather than being predefined. SotA models achieve impressive results in classification of low maintainability files. [26] achieved an 82% F1 score with an AdaBoost classifier, while [23] achieved 93% F1 with a random forest classifier. However, both approaches share a critical limitation—they rely on human experts to create a ground truth of annotated data. PROMISE [27] and MainData [28] were the datasets used in these studies, relying on expert annotation. This introduces subjectivity and limits scale. These datasets are also by design older and cannot reflect the current coding landscape.

This limitation is the key motivation for an algorithmic approach to labelling, allowing us to create a dataset based on current codebases at scale.

A crucial step in applying these machine learning to maintainability detection is the transformation of source code into a machine-readable representation. Abstract Syntax Trees (ASTs) provide this structure, encoding the syntactic and hierarchical relationships between program elements. By traversing or analysing the AST, a wide range of numerical and categorical features can be extracted: such as nesting depth, branching density, or average method size, which serve as the input features for machine learning models [29]. Various learning models can then be trained on these AST-derived metrics to classify code fragments according to maintainability characteristics.

3.5 Summary

The machine learning approach offers a compelling alternative to expert-derived static analysis rules. By using interpretable models such as Decision Trees, we can extract human-readable rules that map directly to executable code quality checks. Random Forests provide higher accuracy but require additional effort to extract interpretable rules. Prior work has demonstrated strong classification performance (82-93% F1), but these approaches rely on expert-labeled datasets that are difficult to scale and may reflect outdated coding standards.

This study takes a different approach: rather than relying on human experts to label data, we use the Hub Score—an objective metric derived from version control metadata —to automatically label files as "High Risk" or "Low Risk." This automated labeling enables us to train machine learning models on a larger, more diverse dataset without the overhead of manual expert annotation. The resulting model learns which structural characteristics (nesting depth, complexity, coupling) correlate with files that developers actually change frequently, translating these patterns into static analysis rules that are empirically grounded rather than opinion-based.

4 Methodology

This study follows an empirical, data-driven pipeline to derive static analysis rules from real-world research software. The process is divided into four primary phases. First, we aggregate a dataset of C++ repositories from the Journal of Open Source Software (JOSS). Second, we employ a custom-built tool to extract evolutionary coupling metrics and calculate a Hub Score for each file, providing an objective "High Risk" or "Low Risk" label. Third, these labeled files are used to train a Decision Tree model to identify the structural characteristics of high-risk code. Finally, the logical paths within the trained model are translated into human-readable static analysis rules, bridging the gap between historical developer behavior and proactive code quality standards.

4.1 Dataset Selection and Acquisition

The primary goal of this paper is to derive objective rules to raise the level of coding standards in the research software scene. This lends itself to using research software as the dataset. JOSS [30] an open source website which allows for researchers to submit open source research project, this is a dataset which provides a curated set of peer-reviewed research software, ensuring domain-specific relevance to use as each repository is guaranteed to be a research repo. To ensure sufficient architectural complexity, projects were filtered using a purposive sampling strategy: a minimum of 4 collaborators and a commit history between 100 and 5,000 commits. C++ was selected as the target language due to its prevalence in high-performance research computing and the inherent risk associated with its low-level memory operations. Thus creating a dataset which comprises sufficiently complex projects all of which have academic grounding for evaluation.

4.2 Data Labeling

To categorize the dataset,automated labeling heuristics were utilised rather than traditional expert manual review. While expert opinion is often the standard for establishing "ground truth," it is difficult to scale across thousands of research repositories. Instead, we analyzed the historical behavior of each file to assign a Hub Score: a composite metric that measures a file's "entanglement" within the project. This score is calculated based on three key factors: the number of coupled files, the average coupling ratio, and the file's overall code churn.We gathered these metrics by running a custom analysis tool over the entire commit history of each repository. The tool constructs a Co-change Graph where, Nodes represent individual files and Edges represent shared commits between files. The graph allows us to evaluate the coupling of each file by comparing the number of times it was modified alongside other files versus the number of times it was modified in isolation.

Drawing on three complementary insights from prior work, we propose a composite Hub Score that synthesizes temporal coupling, file entanglement, and change frequency into a single normalized metric. [14] demonstrated that commit coefficients between files serve as reliable indicators of architectural debt—files that frequently change together

despite lacking structural dependencies represent "Modularity Violations" that accumulate maintenance burden. Building on this, [15] showed that files exhibiting high code churn (frequent additions/deletions) correlate directly with reduced maintainability, as each change introduces potential for new defects. Finally, [12] empirically validated that coupling density—measured by the number of partner files—directly increases maintenance overhead across 1,200 Google repositories.

As noted by [14], focusing primarily on modularity violations ignores other potential risk factors. However, this approach is justified by the unique scale of our study. Unlike traditional, manually-curated datasets such as PROMISE [27] or MainData [28], our automated "broad-spectrum" filtering allows us to identify the most critical offenders across a significantly larger volume of data. By combining these repositories, we created a dataset that is both objective and massive in scope, providing a high-quality surface area for supervised learning using "High Risk" and "Low Risk" labels.== Feature Selection. To quickly calculate a relative score of each file this formula was used to calculate a value called a Hub Score which represents the temporal coupling and change rate of every file in compared to others.

4.2.1 Hub Score

$$\text{HubScore} = |(C) \times \left(\frac{P}{F_t}\right) \times \left(\frac{W_f}{W_t}\right) \tag{2}$$

Variable Definitions

- $|(C)$ (**avg_coupling**): The mean strength or frequency of dependencies between this file and its partners.
- P (**partner_count**): The number of unique files that this specific file is coupled with.
- F_t (**total_files**): The total number of files in the repository or local subset.
- W_f (**file_churn**): The number of changes or commits associated with this specific file.
- W_t (**total_churn**): The total sum of changes across the entire repository.

This is a bespoke formula normalised to allow accurate comparison across repositories. The core justification to use this formula is derived from three core principles:

1. A logic file should not have to change often — supported by the Open Closed principle and [15], which demonstrated that high frequency and large size of changes correlate to an increased maintenance overhead.
2. A logic file should not be highly coupled to many partner files — [12] empirically validated that coupling density directly increases maintenance overhead across 1,200 Google repositories.
3. Files that change together despite lacking structural dependencies represent hidden architectural debt — [14] identified these "Modularity Violations" as the most common and expensive form of architectural debt, where files co-change without explicit dependencies.

This three-principle justification is not merely theoretical; it mirrors the approach of prior work that uses evolution history as ground truth for supervised learning. demonstrated

that co-change information can produce higher-quality labels for code smell detection than expert annotation alone, validating the use of temporal coupling as an objective labeling mechanism for machine learning. The formula was further substantiated where when benchmarking against sonarqube high risk files identified by sonarqube were also identified by this formula, while not all high risk files were identified, each of the high risk files identified were associated with an above average maintenance burden which evidences the ability of it.

4.2.2 Contrastive sampling

To prepare the data for a binary Decision Tree classifier, the continuous Hub Score was split into two categorical classes: High Risk and Low Risk. Rather than utilizing a static numerical threshold, which may vary significantly across project scales, a Global Rank-Ordering strategy was applied:High Risk Class ($Y = 1$): Defined as the top 2,000 files ($N_{\{\top\}}$) across the entire processed corpus. These files represent the extreme outliers in terms of coupling and change frequency. Low Risk Class ($Y = 0$): To ensure a clear decision boundary for the classifier, we utilized Contrastive Under-sampling. We sampled an equivalent $N = 2,000$ files from the bottom quartile of the Hub Score distribution.By excluding the "middle-ground" files (those in the 50th percentile), we maximize the variance between classes, allowing the Decision Tree to more effectively identify the structural signatures of high-maintenance code.

4.2.3 Addressing Data Leakage

A critical consideration in this methodology is the separation of Labeling Criteria and Training Features. The Labels are derived from Temporal Metadata (commit history, co-change frequencies, and historical churn). The Features (inputs for the Decision Tree) are derived strictly from Static Source Code Analysis (e.g., McCabe Complexity, LoC, Nesting Depth). This separation ensures that the model is learning to predict future maintenance risk based on the current state of the code, rather than simply "re-discovering" the components of the Hub Score formula.

4.2.4 Dataset Summary

The final balanced dataset consists of 4,000 observations. This 50/50 class distribution prevents the classifier from developing a majority-class bias, ensuring that the resulting decision rules are equally sensitive to the characteristics of "Hub" files.

4.3 Model Implementation and Training

4.4 Rule Derivation Process

4.5 Evaluation Framework

Bibliography

[1] G. Wilson *et al.*, "Best Practices for Scientific Computing," *PLoS Biology*, vol. 12, no. 1, p. e1001745, Jan. 2014, doi: 10.1371/journal.pbio.1001745.

[2] A. Trisovic, M. K. Lau, T. Pasquier, and M. Crosas, "A large-scale study on research code quality and execution," *Scientific Data*, vol. 9, p. 60, Feb. 2022, doi: 10.1038/s41597-022-01143-6.

[3] B. Johnson, Y. Song, E. Murphy-Hill, and R. Bowdidge, "Why don't software developers use static analysis tools to find bugs?," in *2013 35th International Conference on Software Engineering (ICSE)*, May 2013, pp. 672–681. doi: 10.1109/ICSE.2013.6606613.

[4] M. Haddouchi and A. Berrado, "Forest-ORE: Mining an optimal rule ensemble to interpret random forest models," *Engineering Applications of Artificial Intelligence*, vol. 143, p. 109997, Mar. 2025, doi: 10.1016/j.engappai.2024.109997.

[5] G. Wilson, "Software Carpentry: Getting Scientists to Write Better Code by Making Them More Productive," *Computing in Science & Engineering*, vol. 8, no. 6, pp. 66–69, Nov. 2006, doi: 10.1109/MCSE.2006.122.

[6] R. C. Jiménez *et al.*, "Four simple recommendations to encourage best practices in research software." Accessed: Feb. 07, 2026. [Online]. Available: https://f1000research.com/articles/6-876

[7] B. Marwick, "Computational Reproducibility in Archaeological Research: Basic Principles and a Case Study of Their Implementation," *Journal of Archaeological Method and Theory*, vol. 24, no. 2, pp. 424–450, June 2017, doi: 10.1007/s10816-015-9272-9.

[8] S. J. Eglen *et al.*, "Towards standard practices for sharing computer code and programs in neuroscience," *Nature neuroscience*, vol. 20, no. 6, pp. 770–773, May 2017, doi: 10.1038/nn.4550.

[9] J. Howison and J. D. Herbsleb, "The sustainability of scientific software:," *2014*, 2014, doi: https://james.howison.name/pubs/HowisonHerbsleb-Sustainability.pdf.

[10] "ISO 25010." Accessed: Feb. 02, 2026. [Online]. Available: https://iso25000.com/index.php/en/iso-25000-standards/iso-25010

[11] D. L. Parnas, "On the criteria to be used in decomposing systems into modules," *Commun. ACM*, vol. 15, no. 12, pp. 1053–1058, Dec. 1972, doi: 10.1145/361598.361623.

[12] Y. Cai *et al.*, "Understanding Architectural Complexity, Maintenance Burden, and Developer Sentiment -A Large-Scale Study," in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, Apr. 2025, pp. 2176–2187. doi: 10.1109/ICSE55347.2025.00168.

[13] P. Oman and J. Hagemeister, "Metrics for assessing a software system's maintainability," in *Proceedings Conference on Software Maintenance 1992*, Nov. 1992, pp. 337–344. doi: 10.1109/ICSM.1992.242525.

[14] L. Xiao, Y. Cai, R. Kazman, R. Mo, and Q. Feng, "Identifying and quantifying architectural debt," in *Proceedings of the 38th International Conference on Software Engineering*, Austin Texas: ACM, May 2016, pp. 488–498. doi: 10.1145/2884781.2884822.

[15] C. Faragó, P. Hegedűs, and R. Ferenc, "Cumulative code churn: Impact on maintainability," in *2015 IEEE 15th International Working Conference on Source Code Analysis and Manipulation (SCAM)*, Sept. 2015, pp. 141–150. doi: 10.1109/SCAM.2015.7335410.

[16] T. Bakota, P. Hegedüs, P. Kortvelyesi, R. Ferenc, and T. Gyimothy, "A probabilistic software quality model," Sept. 2011, pp. 243–252. doi: 10.1109/ICSM.2011.6080791.

[17] A. Gosain and G. Sharma, "Static Analysis: A Survey of Techniques and Tools," in *Intelligent Computing and Applications*, D. Mandal, R. Kar, S. Das, and B. K. Panigrahi, Eds., New Delhi: Springer India, 2015, pp. 581–591. doi: 10.1007/978-81-322-2268-2_59.

[18] S. Johnson and M. Hill, "Lint, a C Program Checker," 1978. Accessed: Feb. 08, 2026. [Online]. Available: https://www.semanticscholar.org/paper/Lint%2C-a-C-Program-Checker-Johnson-Hill/74617cffa3c6438d04aa99bef1cca415de47d0d3

[19] E. Dietrich, "How Has Static Code Analysis Changed Through the Years?." Accessed: Feb. 22, 2026. [Online]. Available: https://blog.ndepend.com/static-code-analysis-changed-years/

[20] E. J. Weyuker, "Empirical Software Engineering Research - The Good, The Bad, The Ugly," in *2011 International Symposium on Empirical Software Engineering and Measurement*, Sept. 2011, pp. 1–9. doi: 10.1109/ESEM.2011.66.

[21] M. Borg, M. Ezzouhri, and A. Tornhill, "Ghost Echoes Revealed: Benchmarking Maintainability Metrics and Machine Learning Predictions Against Human Assessments." Accessed: Mar. 16, 2026. [Online]. Available: http://arxiv.org/abs/2408.10754

[22] T. M. Khoshgoftaar and E. B. Allen, "Controlling Overfitting in Classification-Tree Models of Software Quality," *Empirical Software Engineering*, vol. 6, no. 1, pp. 59–79, Mar. 2001, doi: 10.1023/A:1009803004576.

[23] I. Bluemke and P. Borsukiewicz, "Experiments on software error prediction using Decision Tree and Random Forest algorithms," pp. 865–869, Sept. 2023, doi: 10.15439/2023F363.

[24] J. R. Quinlan, *C4.5: programs for machine learning.* in The Morgan Kaufmann series in machine learning. San Mateo, Calif: Morgan Kaufmann Publishers, 1993.

[25] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, Oct. 2001, doi: 10.1023/A:1010933404324.

[26] S. Bertrand, S. Ciappelloni, P.-A. Favier, and J.-M. André, "Replication and Extension of Schnappinger's Study on Human-level Ordinal Maintainability Prediction Based on Static Code Metrics," *Proceedings of the 27th International Conference on Evaluation and Assessment in Software Engineering*, pp. 241–246, June 2023, doi: 10.1145/3593434.3593488.

[27] G. Boetticher, T. Menzies, and T. Ostrand, "\{PROMISE\} Repository of empirical software engineering data," Jan. 2007.

[28] M. Schnappinger, A. Fietzke, and A. Pretschner, "Defining a Software Maintainability Dataset: Collecting, Aggregating and Analysing Expert Evaluations of Software Maintainability," in *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, Sept. 2020, pp. 278–289. doi: 10.1109/ICSME46990.2020.00035.

[29] S. Bertrand, P.-A. Favier, and j.-m. Andre, "Building an Operable Graph Representation of a Java Program as a Basis for Automatic Software Maintainability Analysis," June 2022, pp. 243–248. doi: 10.1145/3530019.3534081.

[30] "Journal of Open Source Software." Accessed: Apr. 02, 2026. [Online]. Available: https://joss.theoj.org/